

(a) Calculate the transmission delay for a single packet on one link [1 Marks]

Working: The transmission delay d_{trans} for a packet of size L over a link with rate R is given by: $d_{\text{trans}} = L/R$ Given: $L = 1000$ bits, $R = 2 \times 10^6$ bits/s: $d_{\text{trans}} = 1000 \text{ bits} / 2 \times 10^6 \text{ bits/s} = 0.0005 \text{ s} = 0.5 \text{ ms}$	
Transmission Delay (in ms)	0.5ms

(b) Determine the total end-to-end delay for the first packet to reach Host B [2 Marks].

Working: The first packet travels across 3 links with 2 intermediate routers. Using the general formula for the first packet's end-to-end delay over N links: $D_{\text{first packet}} = N \times d_{\text{prop}} + N \times d_{\text{trans}} + (N-1) \times d_{\text{proc}}$ $D_{\text{first packet}} = 3 \times 5 \text{ ms} + 3 \times 0.5 \text{ ms} + (3-1) \times 1 \text{ ms}$ $D_{\text{first packet}} = 15 \text{ ms} + 1.5 \text{ ms} + 2 \text{ ms} = 18.5 \text{ ms}$	
Total End-To-End Delay (in ms)	18.5ms

(c) Calculate the total time required for the entire file to be transmitted from Host A to Host B [2 Marks].

Working: First, calculate the total number of packets: $N_p = F/L = (10 \times 10^6 \text{ bits}) / 1000 \text{ bits} = 10000 \text{ packets}$ In a pipelined scenario, the total time is: $T_{\text{total}} = D_{\text{first packet}} + (N_p - 1) \times d_{\text{trans}}$	
--	--

$T_{\text{total}} = 18.5 \text{ ms} + (10000 - 1) \times 0.5 \text{ ms}$ $T_{\text{total}} = 18.5 \text{ ms} + 4999.5 \text{ ms} = 5018 \text{ ms} = 5.018 \text{ s}$	
Total Time Required (in secs)	5.018s

(d) If the file were transmitted as a single, unsegmented message (i.e., one large packet of size F), what would be the total end-to-end delay [2 Marks]?

Working: For a single large packet: $L_{\text{single}} = F = 10 \times 10^6 \text{ bits}$ $d_{\text{trans, single}} = 10 \times 10^6 \text{ bits} / 2 \times 10^6 \text{ bits/s} = 5 \text{ s}$ $D_{\text{single packet}} = 3 \times 5 \text{ ms} + 3 \times 5 \text{ s} + 2 \times 1 \text{ ms}$ $D_{\text{single packet}} = 15 \text{ ms} + 15 \text{ s} + 2 \text{ ms} = 15.017 \text{ s}$	
Total End-to-End Delay (in secs)	15.017s

Q2: A client resolves the hostname www.example.com.

Iterative DNS resolution involves:

- Local DNS query = **2 ms**
- Root server, TLD server, Authoritative server: each requires **1 RTT = 60 ms** from the local DNS server.
- TCP connection to web server after DNS: **1 RTT = 60 ms**
- Web object size = **500 KB**
- Download link = **8 Mbps**

(a) Compute the DNS resolution delay when cache is cold (requires all 3 lookups) [2 Mark].

Working: Local DNS query = 2 ms Root lookup = 60 ms, TLD lookup = 60 ms, Authoritative lookup = 60 ms $D_{\text{DNS}} = 2 + 60 + 60 + 60 = 182 \text{ ms}$	
---	--

DNS Resolution Delay (in ms)	182ms
------------------------------	-------

(b) Compute the total time to fetch the web page (including DNS + TCP setup + download) [3 Marks].

Working (1KB = 1000bytes): DNS = 182 ms TCP connect = 60 ms Transmission = $4000/8 = 500$ ms Total = 182 (DNS) + 60 (TCP handshake) + 60 (HTML request) + 500 (file transmission) = 802 ms Working (1KB = 1024bytes): DNS = 182 ms TCP connect = 60 ms Transmission = 512 ms Total = 182 (DNS) + 60 (TCP handshake) + 60 (HTML request) + 512 (file transmission) = 814 ms	
Total Time (in ms)	802ms (1KB=1000bytes) 814ms (1KB=1024bytes)

(c) If local DNS has cached the IP, compute the improved fetch time. What is the percentage improvement [3 Marks]?

Working (1KB = 1000bytes): Local DNS query = 2 ms TCP connect = 60 ms Transmission = 500 ms Total = 2 (DNS) + 60 (TCP handshake) + 60 (HTML request) + 500 (file transmission) = 622 ms Working (1KB = 1024bytes): Local DNS query = 2 ms TCP connect = 60 ms Transmission = 512 ms Total = 2 (DNS) + 60 (TCP handshake) + 60 (HTML request) + 512 (file transmission) = 634 ms	
---	--

With Cache (in ms)	622ms (1KB=1000 bytes) 634ms(1KB=1024 bytes)
Percentage Improvement	$(802-622)/802 * 100 = 22.4\%$ (1KB=1000bytes) $(814-634)/814 * 100 = 22.1\%$ (1KB=1024bytes)

Question 3

Solution

(a) Difference between accept() and connect() [3 Marks]

accept	connect
○ Used by the server. ○ Blocks until a client requests a connection. ○ Creates and returns a new socket descriptor, dedicated to communication with that specific client. ○ The original listening socket remains open for further incoming connections.	○ Used by the client. ○ Actively attempts to establish a connection with the server's listening socket. ○ Does not create a new socket; instead, it binds the existing client socket to the server's endpoint.

The server must handle multiple clients simultaneously. If it reused the listening socket, new incoming clients would be blocked. Therefore, accept () spawns a **new communication channel** (new socket descriptor) per client, while the listening socket continues to wait for others. On the client side, there's only **one active connection per socket**, so no need to return a new one.

(b)

1. Why a new socket descriptor is returned instead of reusing the listening socket [2 Marks]

The listening socket is like a “doorbell.” Its job is only to **listen for incoming requests**. Once a request arrives, the server needs a **separate channel** (new socket descriptor) for actual data communication. If the listening socket were reused, it could no longer accept new clients while serving the current one.

2. How the server distinguishes multiple clients [2 Marks]

Each client connection is uniquely identified by the **4-tuple**: (Server IP, Server Port, Client IP, Client Port) Even if many clients connect to the same server IP/port, their **source port numbers differ**. The server uses these differences (recorded in the `sockaddr_in` structure returned by `accept()`) to manage multiple concurrent clients.

(c) Complete TCP Server Code [8 Marks]

Expected Filled Version

```
#include <stdio.h>

#include <stdlib.h>

#include <string.h>

#include <unistd.h>

#include <arpa/inet.h>


int main() {

    int sockfd, new_sock;

    struct sockaddr_in server_addr, client_addr;

    socklen_t addr_len;

    char buffer[50];


    // Step 1: Create socket

    sockfd = socket(AF_INET, SOCK_STREAM, 0);

    if (sockfd == -1) {

        perror("Socket creation failed");

        exit(1);

    }


    // Step 2: Set server address structure
```

```
server_addr.sin_family = AF_INET;
server_addr.sin_addr.s_addr = inet_addr("127.0.0.1");
server_addr.sin_port = htons(8080);

// Step 3: Bind socket
if (bind(sockfd, (struct sockaddr*)&server_addr, sizeof(server_addr)) == -1) {
    perror("Bind failed");
    close(sockfd);
    exit(1);
}

// Step 4: Listen
if (listen(sockfd, 5) == -1) {
    perror("Listen failed");
    close(sockfd);
    exit(1);
}

addr_len = sizeof(client_addr);
new_sock = accept(sockfd, (struct sockaddr*)&client_addr, &addr_len);
if (new_sock == -1) {
    perror("Accept failed");
    close(sockfd);
    exit(1);
}

// Read a single character from client
```

```

read(new_sock, buffer, 1);

// Send back "Welcome"
write(new_sock, "Welcome", 7);

close(new_sock);
close(sockfd);
return 0;
}

```

Q4: Fill in the table by choosing the most suitable option from given MCQS

[30 marks]

1.	B	2.	B	3.	C	4.	B	5.	B
6.	C	7.	B	8.	C	9.	A	10.	B
11.	C	12.	A	13.	B	14.	B	15.	B
16.	C	17.	A	18.	B	19.	C	20.	C
21.	B	22.	D	23.	B	24.	A	25.	D
26.	C	27.	A	28.	B	29.	B	30.	A